Formulários: construir para entender

Caderno do estudante · revisão didática de 22/10/2026

Prof. Fabrício Rodrigues · Curso Técnico em Informática

Um projeto que cresce

Começamos em 23/10 com uma página sem campos e construímos o chamado de suporte. A mesma pasta recebe CSS em 30/10, coleta em 06/11, validação em 13/11, estudo do envio em 27/11 e campo condicional em 04/12. Em 11/12 há avaliação com nova base; 18/12 é devolutiva e recuperação. Sem aula em 20/11.

Como estudar

Leia o objetivo, observe a figura, localize a alteração, insira o trecho e salve. Faça uma previsão antes do teste e explique a diferença entre o previsto e o observado. As figuras são exemplos de interface com dados fictícios; o texto ao lado informa o resultado esperado.

Abrir e entregar

Extraia a base de três arquivos e abra a pasta no VS Code com Live Server. Nas aulas 03 a 07, prefira continuar seu projeto; a base da data é apoio. Registre os testes no portal, baixe o TXT e entregue-o junto do ZIP do código. Os projetos não enviam dados a um servidor.

Percurso

Data	Aprendizagem
16/10	O que já sabemos sobre a web?
23/10	Formulários: fundamentos e campos HTML
30/10	Um formulário legível em qualquer tela
06/11	Os dados aparecem na página
13/11	Mensagens que ajudam a corrigir
27/11	O caminho dos dados
04/12	Um formulário que reage ao contexto
11/12	Avaliação prática: inscrição em oficina
18/12	Corrigir, recuperar e concluir

AULA 02 · 23/10/2026

Formulários: fundamentos e campos HTML

Finalidade, estrutura, controles e validação nativa

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Explicar para que serve um formulário e escolher os dados necessários.

• Associar rótulos e distinguir id, name, type e value.

• Construir escolhas únicas, múltiplas e campos com regras nativas.

Ponto de partida

Abra o projeto-base de 23/10 no VS Code com Live Server. Ele já tem título, estilos e apoio de simulação, mas ainda não tem formulário nem campos. Vamos construir juntos em index.html. Continue esta mesma pasta nas aulas seguintes; os ZIPs por data são bases de apoio para recuperar o ponto de partida.

Para que serve um formulário?

Um formulário reúne informações que uma pessoa fornece para uma tarefa: pesquisar, entrar em uma conta, fazer uma inscrição ou pedir suporte. O controle deve combinar com o dado esperado.

Nosso caso é um chamado de suporte ao laboratório. Vamos usar dados fictícios. Neste primeiro encontro, o navegador confere regras HTML e mostra uma mensagem local; não cria um chamado real.

Tags, atributos e organização

Tags criam os elementos: form, label, input, select, option, textarea, fieldset, legend e button. Atributos configuram esses elementos: id, for, name, type, value e required.

O for do label corresponde ao id do campo. O name identifica o dado. Textos de orientação e rótulos continuam visíveis; placeholder não substitui label.

A classe campo agrupa visualmente cada rótulo e seu controle. A classe campos organiza o conjunto; CSS não define quais dados são coletados.

Nos grupos de radio e checkbox deste projeto, o input fica dentro do label: essa associação implícita dispensa for nesses exemplos.

Construção passo a passo

Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Criar o formulário

Onde inserir: index.html: substitua o comentário INÍCIO DO FORMULÁRIO dentro de main.

Entenda: form abre e fecha o conjunto. div organiza o layout; não coleta dados.

```
<form id="formulario">
  <div class="campos">
    <!-- CAMPOS: inserir aqui -->
  </div>
  <!-- GRUPOS: inserir aqui -->
  <!-- BOTÃO: inserir aqui -->
</form>
```

Resultado esperado: Um contorno branco, ainda sem campos.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Confira que existe um único form, com abertura e fechamento.

2 · Identificar quem pede ajuda

Onde inserir: index.html: antes do comentário CAMPOS, dentro de div.campos. Mantenha o comentário para inserir os próximos campos.

Entenda: label identifica. for = id. name dá nome ao dado. input não tem tag de fechamento.

```
<div class="campo">
  <label for="solicitante">Solicitante fictício (obrigatório)</label>
  <input id="solicitante" name="solicitante"
    type="text" required minlength="2" maxlength="60"
    autocomplete="off">
</div>
```

Resultado esperado: Um rótulo e uma caixa para texto curto.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Clique no rótulo: o cursor deve ir para a caixa.

3 · Escolher o tipo adequado

Onde inserir: index.html: depois do solicitante, antes do comentário CAMPOS.

Entenda: type="text": código PC-07. type="email": formato de endereço. required é atributo.

```
<div class="campo">
  <label for="equipamento">Equipamento (obrigatório)</label>
  <input id="equipamento" name="equipamento" type="text"
    required minlength="2" maxlength="20" placeholder="Ex.: PC-07"
    aria-describedby="ajuda-equipamento">
  <p class="ajuda" id="ajuda-equipamento">Use um código, não uma quantidade.</p>
</div>
<div class="campo">
  <label for="contato">E-mail fictício (obrigatório)</label>
  <input id="contato" name="contato" type="email" required
    autocomplete="off" placeholder="estudante@example.com">
</div>
```

Resultado esperado: Duas novas caixas; o equipamento aceita letras e números.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Digite PC-07 e um e-mail fictício. O teste de formato será feito com o botão na etapa 8.

4 · Oferecer opções com select

Onde inserir: index.html: após contato, antes do comentário CAMPOS.

Entenda: select contém option. A opção inicial tem value=""; required exige outra escolha.

```
<div class="campo">
  <label for="categoria">Categoria (obrigatório)</label>
  <select id="categoria" name="categoria" required>
    <option value="">Escolha uma opção</option>
    <option value="hardware">Hardware</option>
    <option value="software">Software</option>
    <option value="rede">Rede</option>
    <option value="outro">Outro</option>
  </select>
</div>
```

Resultado esperado: Uma lista com quatro categorias e uma instrução inicial.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Abra a lista e escolha Rede. Em 23/10, Outro é apenas uma opção; não abre campo extra.

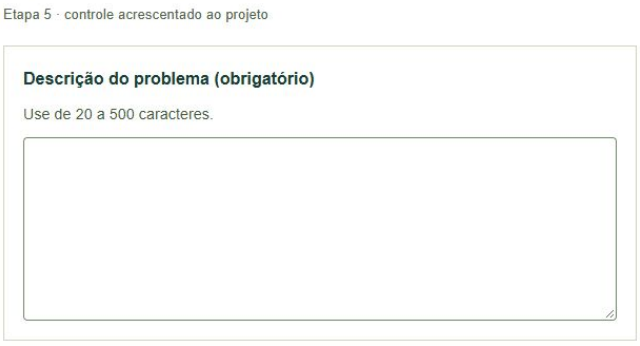
5 · Dar espaço para a descrição

Onde inserir: index.html: depois da categoria, antes do comentário CAMPOS.

Entenda: textarea abre e fecha. rows altera a altura inicial. minlength e maxlength definem limites.

```
<div class="campo largo">
  <label for="descricao">Descrição do problema (obrigatório)</label>
  <p class="ajuda" id="ajuda-descricao">Use de 20 a 500 caracteres.</p>
  <textarea id="descricao" name="descricao" rows="5"
    required minlength="20" maxlength="500"
    aria-describedby="ajuda-descricao"></textarea>
</div>
```

Resultado esperado: Uma caixa de várias linhas com instrução de tamanho.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Escreva uma frase que descreva um defeito fictício. O limite será testado após a etapa 8.

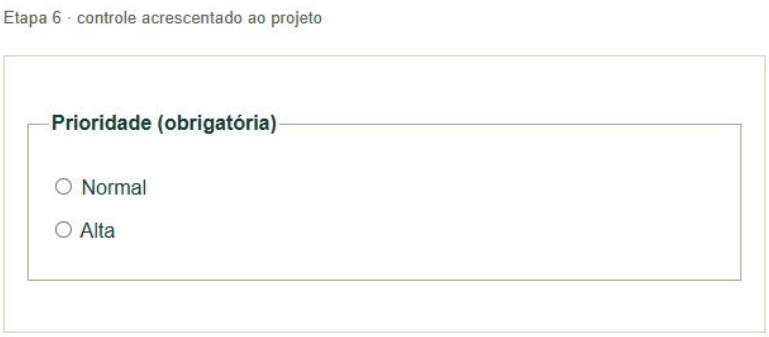
6 · Marcar uma única prioridade

Onde inserir: index.html: depois do fechamento de div.campos e antes do comentário GRUPOS.

Entenda: fieldset agrupa, legend dá título. Radios com o mesmo name formam uma escolha única. Aqui o input fica dentro do label: associação implícita.

```
<fieldset>
  <legend>Prioridade (obrigatória)</legend>
  <label><input type="radio" name="prioridade"
    value="normal" required> Normal</label>
  <label><input type="radio" name="prioridade"
    value="alta"> Alta</label>
</fieldset>
```

Resultado esperado: Duas alternativas dentro do grupo Prioridade.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Marque Normal e depois Alta: a primeira deve ser desmarcada.

7 · Aceitar mais de um sintoma

Onde inserir: index.html: após o fieldset de prioridade e antes do comentário GRUPOS.

Entenda: Checkboxes são independentes. O mesmo name permite coletar uma lista de sintomas.

```
<fieldset>
  <legend>Sintomas (opcionais)</legend>
  <label><input type="checkbox" name="sintomas"
    value="Lentidão"> Lentidão</label>
  <label><input type="checkbox" name="sintomas"
    value="Mensagem de erro"> Mensagem de erro</label>
</fieldset>
```

Resultado esperado: Duas caixas que podem permanecer marcadas juntas.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Marque as duas; depois desmarque uma. Nenhum sintoma é obrigatório.

8 · Testar o formulário

Onde inserir: index.html: substitua o comentário BOTÃO, ainda dentro de form, depois dos grupos.

Entenda: button type="submit" dispara a tentativa. O apoio em script.js impede o envio real.

```
<div class="acoes">
  <button id="simular" type="submit" disabled>
    Testar campos
  </button>
</div>
```

Resultado esperado: O botão fica habilitado pelo script fornecido após salvar a página.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Teste vazio, e-mail inválido e dados válidos. O resultado válido é uma mensagem local, sem resumo ainda.

Prática individual

1. Extraia Aula-02-Projeto.zip, abra a pasta no VS Code e execute index.html com Live Server. Confirme que a página ainda não tem campos.

2. Mantenha o navegador ao lado do editor. Acompanhe as oito etapas: leia a finalidade, insira o trecho no lugar indicado, salve e confira o resultado.

3. Ao concluir os campos, teste vazio, formato de e-mail, limites da descrição, prioridade e sintomas. Use dados fictícios.

4. Registre os testes na ficha da aula 02. Baixe o TXT, compacte somente index.html, style.css e script.js e entregue os dois arquivos na aula.

Casos de teste

Caso	Resultado esperado
Campos obrigatórios vazios	O navegador impede a simulação.
Contato sem formato de e-mail	O campo de contato fica inválido.
Descrição digitada com menos de 20 caracteres	O navegador pede completar o texto.
Clique no rótulo do equipamento	O foco vai para o campo correspondente.
Marcar duas prioridades em sequência	Só a última permanece marcada.
Marcar os dois sintomas	As duas opções permanecem marcadas.
Preenchimento válido	Mensagem local; nenhuma navegação de envio.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual é o papel de name? Qual teste mostrou que seu formulário estava funcionando?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

MDN: seu primeiro formulário

MDN: controles nativos de formulário (em inglês)

W3C WAI: rótulos dos controles (em inglês)

MDN: tipos de input (em inglês)

W3C WAI: agrupamento de controles (em inglês)

MDN: validação de formulários

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 03 · 30/10/2026

Um formulário legível em qualquer tela

CSS, responsividade, instruções e navegação pelo teclado

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Organizar os campos em uma e duas colunas sem rolagem horizontal.

• Manter rótulos, instruções e foco legíveis.

• Testar teclado e ampliação sem depender somente da aparência.

Ponto de partida

Continue a mesma pasta do chamado de suporte criada em 23/10. Faça uma cópia de segurança antes de alterar. Se precisar recuperar o ponto de partida, extraia a base de apoio desta data em outra pasta. Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Layout começa pela tela estreita

Abra a base com o HTML completo da aula anterior. A regra global box-sizing inclui borda e preenchimento na largura calculada. Evite campos com largura fixa maior que a tela.

Uma grade com gap organiza os campos em uma coluna. O código fornecido já mantém inputs com largura de 100% e min-width:0. O objetivo da aula é completar o layout sem alterar os names e as regras HTML.

```
* { box-sizing: border-box; }
.campos {
  display: grid;
  gap: 1.2rem;
}
.campo { min-width: 0; }
```

Duas colunas quando há espaço

A media query entra a partir de 48rem. Até esse limite, a grade continua em uma coluna. O campo de descrição ocupa a linha inteira por meio da classe largo.

O ponto de mudança pertence a este projeto. Ele não define um tipo fixo de aparelho: observe quando os campos deixam de caber com conforto.

```
@media (min-width: 48rem) {
  .campos { grid-template-columns: 1fr 1fr; }
  .largo { grid-column: 1 / -1; }
}
```

Foco visível e ordem de leitura

Tab percorre os controles e Shift+Tab retorna. A ordem do HTML deve acompanhar a ordem de leitura; não use tabindex positivo para reorganizar o formulário.

focus-visible mostra uma indicação clara para navegação. Não remova o contorno sem oferecer indicação equivalente. Rótulos e instruções continuam visíveis durante o preenchimento.

```
:focus-visible {
  outline: 3px solid #6a8f32;
  outline-offset: 4px;
}
```

Teste da experiência

Teste aproximadamente 360px, 768px e 1280px, além da ampliação do navegador em 200%. Compare largura, tamanho dos textos, quebra das instruções e acesso aos botões.

No modo responsivo do navegador, altere a largura e observe o resultado. Medir apenas a quantidade de colunas não comprova que o formulário ficou utilizável.

Construção passo a passo

Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Organizar em uma coluna

Onde inserir: style.css: confira as regras de .campos e .campo.

Entenda: Use display: grid, gap e min-width: 0; mantenha os rótulos visíveis.

```
.campos {
  display: grid;
  gap: 1.2rem;
}
.campo { min-width: 0; }
```

Resultado esperado: Na janela estreita, os campos cabem sem rolagem horizontal.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

2 · Acrescentar duas colunas

Onde inserir: style.css: no final, acrescente a media query de 48rem.

Entenda: Dentro de @media, defina grid-template-columns: 1fr 1fr e .largo com grid-column: 1 / -1.

```
@media (min-width: 48rem) {
  .campos { grid-template-columns: 1fr 1fr; }
  .largo { grid-column: 1 / -1; }
}
```

Resultado esperado: Na janela larga, duas colunas; a descrição ocupa a linha inteira.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

3 · Destacar o foco

Onde inserir: style.css: acrescente :focus-visible antes da media query.

Entenda: Defina outline e outline-offset. Percorra o formulário com Tab.

```
:focus-visible {
  outline: 3px solid #6a8f32;
  outline-offset: 4px;
}
```

Resultado esperado: O campo em foco tem contorno visível e segue a ordem do HTML.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

Prática individual

1. Continue seu chamado de suporte da aula anterior; se precisar, use o ZIP da data como base de apoio. Abra a pasta no VS Code e execute index.html com Live Server.

2. Completar a media query para duas colunas e fazer a descrição ocupar a largura da grade.

3. Acrescentar foco visível e revisar espaçamentos dos grupos e das instruções.

4. Pedir a um colega que percorra o formulário pelo teclado e repetir os testes nas larguras indicadas.

Casos de teste

Caso	Resultado esperado
Tela de aproximadamente 360px	Uma coluna; controles sem corte lateral.
Tela larga a partir de 48rem	Duas colunas; descrição ocupa a linha.
Tab e Shift+Tab	Foco visível, sequência compreensível.
Ampliação de 200%	Conteúdo legível e controles alcançáveis.
Texto longo na instrução	Quebra de linha sem sobrepor outro campo.
Preenchimento inválido	Regras HTML da aula anterior continuam ativas.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual decisão ajudou a tela estreita? Como você conferiu o foco?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

MDN: estilização de formulários (em inglês)

MDN: fundamentos de CSS Grid

W3C WAI: rótulos dos controles (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 04 · 06/11/2026

Os dados aparecem na página

Evento submit, FormData e resumo local com JavaScript

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Escutar submit no formulário e impedir a navegação na simulação.

• Coletar campos por name, preservando múltiplos checkboxes.

• Montar um resumo com elementos do DOM e textContent.

Ponto de partida

Continue a mesma pasta do chamado de suporte criada em 23/10. Faça uma cópia de segurança antes de alterar. Se precisar recuperar o ponto de partida, extraia a base de apoio desta data em outra pasta. Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

O evento pertence ao formulário

Usar submit permite tratar o botão e a submissão pelo teclado. A validação HTML interativa acontece antes do submit: campos inválidos podem impedir que esse evento aconteça.

preventDefault interrompe a ação padrão nesta demonstração. Ele não valida, não envia para um servidor e não grava os dados. A base já oferece um bloqueio de navegação; substitua a mensagem pelo processamento do resumo.

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  const dados = new FormData(form);
  mostrarResumo(dados);
});
```

Coleta por name

new FormData(form) monta os pares de dados dos controles participantes. get recupera um valor; getAll recupera todos os valores com o mesmo name. Checkboxes desmarcados não entram na coleta.

Um campo com id e sem name continua aparecendo na tela, mas não entra como par nome/valor na coleta. Os sintomas selecionados precisam aparecer juntos no resumo.

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Resumo como conteúdo, sem interpretar HTML

Crie dt e dd para cada item da lista de descrição. textContent apresenta o valor como texto. Não use innerHTML para inserir o que a pessoa digitou.

Limpe o resumo anterior antes de montar o novo. Depois torne a seção visível e leve o foco ao título com tabindex=-1. O resumo corresponde à última simulação, não a um registro persistente.

```
function adicionarLinha(titulo, valor) {
  const dt = document.createElement('dt');
  const dd = document.createElement('dd');
  dt.textContent = titulo;
  dd.textContent = String(valor);
  resumo.append(dt, dd);
}
```

Estado do formulário depois da tentativa

Uma falha de validação deve manter os valores para correção. Um novo envio deve substituir o resumo anterior, sem duplicar linhas.

O botão Novo preenchimento é type=button. Ele pode chamar reset, ocultar o resumo e devolver o foco ao primeiro campo. Avise que o projeto perde os dados quando a página é recarregada.

Construção passo a passo

Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Preparar o resultado

Onde inserir: index.html: depois de form, crie section#resultado hidden com h2#titulo-resumo tabindex="-1" e dl#dados-resumo. Dentro de div.acoes do form, acrescente button#novo type="button".

Entenda: Crie a área para o resumo e mantenha Novo preenchimento fora da área escondida.

```
<!-- Depois do fechamento de form: -->
<section class="painel" id="resultado" hidden
  aria-labelledby="titulo-resumo">
  <h2 id="titulo-resumo" tabindex="-1">Resumo da última simulação</h2>
  <p>Nenhuma solicitação foi enviada ou gravada em um servidor.</p>
  <dl id="dados-resumo"></dl>
</section>

<!-- Dentro de div.acoes do form, depois do botão submit: -->
<button type="button" id="novo">Novo preenchimento</button>
```

Resultado esperado: A área do resumo começa oculta; Novo preenchimento permanece acessível.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

2 · Coletar no submit

Onde inserir: script.js: substitua o manipulador de submit fornecido; mantenha um único listener.

Entenda: Cancele a navegação; crie new FormData(form); use get e getAll.

```
const resultado = document.querySelector('#resultado');
const resumo = document.querySelector('#dados-resumo');
// No único listener de submit, após preventDefault():
const dados = new FormData(form);
mostrarResumo(dados);
```

Resultado esperado: Ao marcar dois sintomas, ambos permanecem na lista coletada.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

3 · Mostrar e reiniciar

Onde inserir: script.js: implemente mostrarResumo, adicionarLinha e o clique de #novo conforme os exemplos do conteúdo.

Entenda: Use createElement, textContent e replaceChildren; revele o resultado e leve o foco ao título. Mude o texto do botão submit para Gerar resumo local.

Resultado esperado: Um envio válido mostra resumo local. Novo preenchimento limpa e foca o solicitante.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

Prática individual

1. Continue seu chamado de suporte da aula anterior; se precisar, use o ZIP da data como base de apoio. Abra a pasta no VS Code e execute index.html com Live Server.

2. Substituir o tratamento de submit pela coleta com FormData e pela chamada que monta o resumo.

3. Mostrar solicitante, equipamento, contato, categoria, prioridade, descrição e todos os sintomas selecionados.

4. Acrescentar a limpeza do resumo em uma nova simulação e o botão de novo preenchimento.

Casos de teste

Caso	Resultado esperado
Campo obrigatório vazio	O resumo não aparece.
Preenchimento válido	Resumo local com os valores informados.
Dois sintomas marcados	Ambos aparecem no resumo.
Nenhum sintoma marcado	Mensagem de nenhuma opção selecionada.
Texto com exemplo na descrição	As marcas aparecem como texto, sem formatação HTML.
Duas simulações válidas	O resumo mais recente substitui o anterior.
Novo preenchimento	Campos reiniciados e foco no primeiro campo.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Como o name chega ao FormData? Por que usar textContent no resumo?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

MDN: evento submit (em inglês)

MDN: validação de formulários

MDN: uso de FormData (em inglês)

MDN: textContent (em inglês)

MDN: elemento button (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 05 · 13/11/2026

Mensagens que ajudam a corrigir

Regras complementares, validade personalizada e casos de teste

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Comparar o contato com sua confirmação sem perder as regras nativas.

• Criar e limpar mensagens com setCustomValidity.

• Registrar casos válidos, inválidos e limites da descrição.

Ponto de partida

Continue a mesma pasta do chamado de suporte criada em 23/10. Faça uma cópia de segurança antes de alterar. Se precisar recuperar o ponto de partida, extraia a base de apoio desta data em outra pasta. Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Comparação de dois campos

A base acrescenta o campo de confirmação com type=email e required. Isso confere cada formato, mas não prova que os dois valores são iguais.

A função compara os valores e define uma mensagem quando diferem. String vazia limpa a validade personalizada. Escute input nos dois campos, pois qualquer um pode mudar depois.

```
function conferirContato() {
  const diferente = confirmar.value !== contato.value;
  confirmar.setCustomValidity(
    confirmar.value && diferente ? 'Repita o mesmo e-mail.' : ''
  );
}
contato.addEventListener('input', conferirContato);
confirmar.addEventListener('input', conferirContato);
```

Descrição e limites do teste

minlength e maxlength tratam o comprimento do valor. Nesta aula acrescentamos a regra de pelo menos 20 caracteres depois de trim, para não aceitar uma descrição formada só por espaços.

A função limpa a mensagem quando o texto passa. Teste digitando no campo; restrições de comprimento nativas têm particularidades quando um script atribui o valor diretamente. Não altere o valor do aluno em silêncio.

```
function conferirDescricao() {
  const curta = descricao.value.length > 0
    && descricao.value.trim().length < 20;
  descricao.setCustomValidity(
    curta ? 'Descreva o problema com pelo menos 20 caracteres.' : ''
  );
}
descricao.addEventListener('input', conferirDescricao);
```

Nova tentativa sem mensagens antigas

Antes de montar o resumo, atualize as verificações e consulte reportValidity. Os listeners de input são essenciais para limpar um erro anterior que bloquearia a próxima submissão.

Ao reiniciar, faça reset e limpe a validade personalizada. Mostrar um aviso não deve apagar os campos nem transferir a culpa para a pessoa. A mensagem precisa indicar como corrigir.

```
form.addEventListener('submit', (event) => {
  event.preventDefault();
  conferirContato();
  conferirDescricao();
  if (!form.reportValidity()) return;
  mostrarResumo(new FormData(form));
});
```

Construção passo a passo

Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Adicionar confirmação

Onde inserir: index.html: depois de contato, ainda dentro de div.campos.

Entenda: Adicione label e input#confirmar name="confirmar" type="email" required.

```
<div class="campo">
  <label for="confirmar">Repita o e-mail fictício (obrigatório)</label>
  <input id="confirmar" name="confirmar" type="email" required
    autocomplete="off">
</div>
```

Resultado esperado: O novo campo pede repetição do contato.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

2 · Comparar e limpar mensagens

Onde inserir: script.js: depois das seleções dos elementos e antes do listener de submit.

Entenda: Crie conferirContato e conferirDescricao; associe eventos input para limpar o erro quando corrigido.

```
const contato = document.querySelector('#contato');
const confirmar = document.querySelector('#confirmar');
const descricao = document.querySelector('#descricao');
// Acrescente as duas funções de conferência mostradas no conteúdo.
```

Resultado esperado: Contatos diferentes pedem correção. Ao igualar, o erro desaparece.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

3 · Conferir antes do resumo

Onde inserir: script.js: substitua o listener de submit; mantenha apenas um.

Entenda: Chame as duas conferências; se reportValidity retornar false, encerre; caso contrário, mostre o resumo.

Resultado esperado: Espaços não completam a descrição. Uma tentativa inválida mantém os valores.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

Prática individual

1. Continue seu chamado de suporte da aula anterior; se precisar, use o ZIP da data como base de apoio. Abra a pasta no VS Code e execute index.html com Live Server.

2. Implementar a comparação e registrar input nos dois e-mails.

3. Acrescentar a regra da descrição com trim e limpar as mensagens quando o valor for corrigido.

4. Repetir a tentativa após cada correção e registrar os limites e o reinício.

Casos de teste

Caso	Resultado esperado
E-mails diferentes	Mensagem de repetir o mesmo contato.
Corrigir o segundo e-mail	A mensagem antiga é limpa.
Mudar o primeiro depois da confirmação	A divergência volta a ser identificada.
Descrição formada por espaços	A regra complementar bloqueia a simulação.
Descrição digitada com 19 caracteres úteis	A simulação fica bloqueada.
Descrição digitada com 20 caracteres úteis	A regra de tamanho permite continuar.
Novo preenchimento após erro	Não permanece mensagem personalizada antiga.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Quando sua mensagem é limpa? Que teste demonstrou a correção do erro anterior?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

MDN: setCustomValidity (em inglês)

MDN: validação de formulários

W3C WAI: notificações e erros (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 06 · 27/11/2026

O caminho dos dados

GET, POST, HTTPS e contrato do projeto integrador

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Distinguir uma simulação local de uma requisição a um serviço.

• Comparar GET e POST e explicar o papel do HTTPS.

• Documentar campos e regras de um serviço futuro sem inventar um backend.

Ponto de partida

Continue a mesma pasta do chamado de suporte criada em 23/10. Faça uma cópia de segurança antes de alterar. Se precisar recuperar o ponto de partida, extraia a base de apoio desta data em outra pasta. Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Os limites do protótipo

Até aqui o JavaScript trabalha somente na página. Mostrar um resumo não faz uma requisição de envio. O GitHub Pages publica os arquivos estáticos do curso; não executa um serviço próprio para receber chamados.

Uma aplicação real precisaria de uma rota de servidor e de regras de recebimento. Nesta aula documentaremos esse contrato e compararemos representações, sem enviar dados a um endereço externo.

GET e os parâmetros da URL

GET costuma ser usado para recuperar informações. Em um formulário com method=get, os pares de dados podem compor a query da URL. Eles podem aparecer no histórico e ser compartilhados junto com o endereço.

O exemplo gera uma string com URLSearchParams e a mostra como texto. exemplo.test é um domínio de exemplo; o projeto não navega para ele. Observe como nomes, espaços e caracteres são representados.

```
const dados = new FormData(form);
const parametros = new URLSearchParams(dados);
const exemplo = 'https://exemplo.test/chamados?'
  + parametros.toString();
// Apenas exibir o texto. Não navegar para esse endereço.
```

POST, HTTPS e conferência no servidor

Em um formulário HTML com method=post, os dados podem seguir no corpo da requisição. POST sozinho não criptografa dados. HTTPS protege o tráfego em trânsito. Não coloque informações sensíveis em uma URL por achar que o método resolveu toda a proteção.

O servidor deve conferir formato, tamanho, escolhas permitidas e demais regras. O usuário pode alterar a página ou enviar outra requisição. A validação HTML e o JavaScript servem à experiência local e precisam de conferência no serviço real.

Contrato do integrador

Documente no HTML um bloco de contrato: finalidade, método escolhido, rota proposta, campos e regras a aplicar no servidor. Identifique claramente que a rota não está implementada.

A base exibe exemplos de URL GET e de um corpo para POST depois da simulação. São representações de estudo. Complete o contrato e confronte seus nomes de campos com o que FormData realmente coleta.

```
<!-- Exemplo de contrato para uma aplicação futura.
Método: POST. Rota proposta: /chamados.
O servidor deve validar os campos novamente.
Este projeto estático não implementa essa rota.
-->
```

Construção passo a passo

Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Separar simulação e servidor

Onde inserir: index.html: acrescente o comentário do contrato proposto antes de form.

Entenda: Registre POST /chamados como proposta futura. Não acrescente action para uma rota inexistente.

```
<!-- Contrato proposto para uma aplicação futura:
  POST /chamados. O servidor deve validar os campos novamente.
  Este projeto estático não implementa essa rota.
-->
```

Resultado esperado: O projeto continua local, sem abrir uma solicitação real.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

2 · Representar GET e POST

Onde inserir: index.html: dentro de #resultado, adicione pre#representacao. script.js: crie representarEnvio e chame-a após mostrarResumo no listener de submit.

Entenda: URLSearchParams representa parâmetros da URL; JSON.stringify representa o corpo proposto.

```
<h3>Representação didática de envio</h3>
<pre id="representacao"></pre>
```

Resultado esperado: O navegador mostra dois textos para comparação; não executa uma requisição.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

3 · Conferir o contrato

Onde inserir: script.js: no objeto de POST, inclua sintomas como array e omita confirmar.

Entenda: Compare campos, método, destino proposto e formato. Limpe os exemplos ao editar ou reiniciar.

```
// No listener de submit, após mostrarResumo(dados):
representarEnvio(dados);
// Nos listeners de input e de #novo, limpe a representação:
document.querySelector('#representacao').textContent = '';
```

Resultado esperado: A representação de POST não duplica confirmar; HTTPS protege o tráfego, não valida os dados.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

Prática individual

1. Continue seu chamado de suporte da aula anterior; se precisar, use o ZIP da data como base de apoio. Abra a pasta no VS Code e execute index.html com Live Server.

2. Comparar onde os valores apareceriam em GET e POST e explicar por que não houve envio real.

3. Completar o contrato no HTML: finalidade, POST, rota proposta /chamados, campos, regras e indicação de serviço ainda não implementado.

4. Conferir se os names coletados correspondem ao contrato e registrar o escopo do integrador.

Casos de teste

Caso	Resultado esperado
Simulação com dados fictícios	Resumo e representações locais, sem envio.
Espaços no nome	URL de exemplo com parâmetros codificados.
Dois sintomas selecionados	Representação de POST preserva a lista.
Contrato sem indicar implementação futura	Revisar para não prometer serviço existente.
POST sem HTTPS no argumento	Explicar que POST não oferece criptografia.
Campo sem name	Identificar a ausência na coleta e corrigir o contrato.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual parte do seu projeto roda no navegador? O que ainda faltaria num serviço real?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

GitHub: funcionamento do Pages

MDN: envio de dados (em inglês)

MDN: HTTPS (em inglês)

MDN: uso de FormData (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 07 · 04/12/2026

Um formulário que reage ao contexto

Campos condicionais, estado e organização das funções

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Mostrar um campo adicional somente quando sua opção for escolhida.

• Coordenar hidden, disabled e required.

• Repetir a coleta e o reinício sem valores de uma opção antiga.

Ponto de partida

Continue a mesma pasta do chamado de suporte criada em 23/10. Faça uma cópia de segurança antes de alterar. Se precisar recuperar o ponto de partida, extraia a base de apoio desta data em outra pasta. Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

A interface responde a uma escolha

O HTML da base já contém o bloco de outra categoria e um input inicialmente disabled. A função de estado deve ler o select e atualizar o conjunto de propriedades.

Registre change no select e chame a função ao iniciar a página. Organize a regra em uma função, para também aplicá-la depois de reset. Não espalhe a mesma decisão em vários handlers.

```
function atualizarOutro() {
  const ativo = categoria.value === 'outro';
  blocoOutro.hidden = !ativo;
  outro.disabled = !ativo;
  outro.required = ativo;
  if (!ativo) outro.value = '';
}
categoria.addEventListener('change', atualizarOutro);
atualizarOutro();
```

Esconder não desativa o controle

hidden altera a apresentação do bloco. Um campo obrigatório somente escondido ainda pode interferir na validação. disabled retira o controle da validação e da coleta normal de dados.

required deve ficar ativo apenas quando o campo é necessário. Ao desativar a opção, o projeto apaga o valor antigo para não sugerir um dado que deixou de pertencer à escolha atual.

```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Conferência do fluxo inteiro

Teste entrar em Outro, tentar sem preencher, informar um detalhe e gerar o resumo. Depois retorne a Hardware e confira que o detalhe não aparece na coleta.

Use Novo preenchimento para conferir reset e reaplicação do estado. Revise a comparação dos e-mails e as mensagens: acrescentar uma condição não deve quebrar as regras anteriores.

Construção passo a passo

Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Adicionar o campo complementar

Onde inserir: index.html: dentro de div.campos, após categoria.

Entenda: Crie div#bloco-outro hidden e input#outra_categoria name="outra_categoria" disabled maxlength="100" com label.

```
<div class="campo" id="bloco-outro" hidden>
  <label for="outra_categoria">Qual outra categoria? (obrigatório quando
    ativo)</label>
  <input id="outra_categoria" name="outra_categoria" type="text"
    maxlength="100" disabled>
</div>
```

Resultado esperado: O campo começa oculto e desativado.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

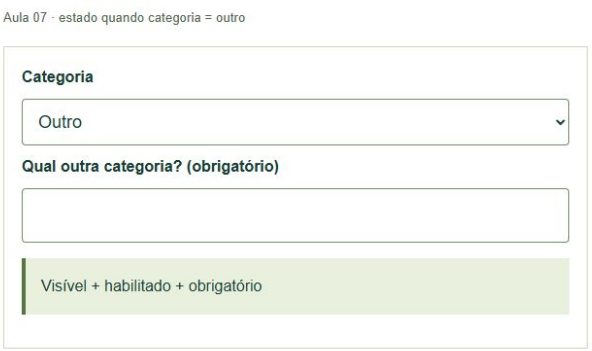
2 · Sincronizar o estado

Onde inserir: script.js: acrescente atualizarOutro e associe change à categoria.

Entenda: Se categoria for outro: hidden=false, disabled=false, required=true. Fora disso: inverta e limpe o valor.

```
const categoria = document.querySelector('#categoria');
const blocoOutro = document.querySelector('#bloco-outro');
const outro = document.querySelector('#outra_categoria');
// Acrescente atualizarOutro e o listener change do conteúdo.
```

Resultado esperado: Outro mostra e exige o campo. Hardware esconde e retira o controle da coleta.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

3 · Repetir o ciclo completo

Onde inserir: script.js: chame atualizarOutro na inicialização e depois de form.reset().

Entenda: Teste Outro → preencher → Hardware → enviar → Novo preenchimento.

```
// Uma vez, após definir atualizarOutro e o listener change:
atualizarOutro();
// No listener de #novo, depois de form.reset():
atualizarOutro();
```

Resultado esperado: Nenhum valor antigo reaparece no resumo; reiniciar restaura o estado inicial.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

Prática individual

1. Continue seu chamado de suporte da aula anterior; se precisar, use o ZIP da data como base de apoio. Abra a pasta no VS Code e execute index.html com Live Server.

2. Criar a função de estado, escutar change e aplicá-la na inicialização.

3. Manter o campo adicional obrigatório apenas em Outro e incluir seu valor no resumo somente quando coletado.

4. Testar todas as transições e executar novamente os casos de validação das aulas anteriores.

Casos de teste

Caso	Resultado esperado
Abertura com categoria vazia	Campo adicional oculto e desabilitado.
Escolher Outro sem detalhar	Campo visível e obrigatório.
Outro com detalhe e dados válidos	Resumo inclui a outra categoria.
Trocar de Outro para Hardware	Bloco oculto e valor antigo limpo.
Gerar resumo após sair de Outro	Não inclui outra_categoria.
Novo preenchimento	Estado inicial reaplicado.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Que diferença existe entre esconder um campo e desabilitá-lo?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

MDN: atributo disabled (em inglês)

MDN: evento submit (em inglês)

MDN: uso de FormData (em inglês)

MDN: setCustomValidity (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 08 · 11/12/2026

Avaliação prática: inscrição em oficina

Aplicação individual dos conhecimentos de formulários

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Construir um formulário para um contexto diferente do chamado.

• Demonstrar estrutura, responsividade, coleta e validação.

• Registrar testes e explicar uma decisão de implementação.

Ponto de partida

Abra a base específica desta data em outra pasta. Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Situação da avaliação

Uma oficina de informática precisa de um protótipo de inscrição. Use somente dados fictícios e mantenha o envio simulado. O professor explicará as regras de consulta e os critérios antes do início.

O participante deve escolher uma oficina (Redes, Hardware, Web ou Outra oficina) e um turno (Manhã ou Tarde). Materiais que levará são opcionais: Caderno e Notebook.

Especificação dos campos

Participante fictício: texto de 2 a 60 caracteres. Contato e confirmação: formato de e-mail, obrigatórios e iguais. Oficina: select obrigatório com opção inicial vazia. Turno: uma opção obrigatória em radio.

Objetivo da participação: de 20 a 500 caracteres úteis. Ao escolher Outra oficina, mostrar Tema sugerido, habilitá-lo e torná-lo obrigatório. Ao sair dessa escolha, limpar e desabilitar esse campo.

Comportamento e entrega

Mostrar um resumo local com os dados da inscrição, preservando os dois materiais opcionais se marcados. Tratar valores como texto. Não enviar nem gravar em servidor.

O formulário deve funcionar em tela estreita e pelo teclado. Manter valores após uma tentativa inválida. O reinício precisa limpar as mensagens e o estado condicional. Entregar os três arquivos em ZIP e o registro TXT.

Construção passo a passo

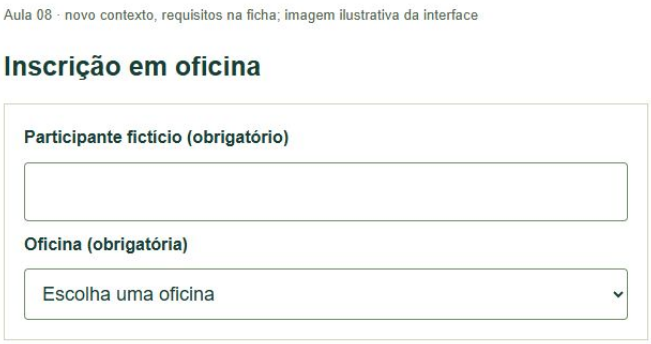
Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Ler antes de implementar

Onde inserir: Abra a base específica de 11/12. Esta é uma nova aplicação: inscrição em oficina.

Entenda: Compare a especificação com os controles necessários; planeje a ordem.

Resultado esperado: Uma base nova permite demonstrar o que você aprendeu no projeto contínuo.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

2 · Construir e testar individualmente

Onde inserir: Edite index.html e script.js; o CSS de apoio está pronto.

Entenda: Implemente os requisitos da ficha e registre também o que não conseguiu concluir.

Resultado esperado: Entregue evidências reais; o professor considera código, comportamento e explicação.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

Critérios propostos da avaliação

Dimensão	Pontos
HTML e controles	2
CSS e acessibilidade de uso	2
Coleta e resumo local	2
Validação e estado condicional	2
Testes e explicação individual	2

Prática individual

1. Ler os requisitos e os critérios com o professor. Baixar a base da avaliação.
2. Construir individualmente o HTML, completar CSS e JavaScript conforme a especificação.
3. Realizar os testes da tabela e registrar os resultados reais, inclusive o que ainda falhou.
4. Entregar ZIP e TXT e explicar um campo, uma regra e um teste ao professor.

Casos de teste

Caso	Resultado esperado
Obrigatórios vazios	Bloqueio da tentativa.
E-mails diferentes e depois corrigidos	Erro comunicado e mensagem limpa ao corrigir.
Outra oficina sem tema	Tema obrigatório.
Trocar de Outra oficina para Web	Tema deixa a validação e a coleta.
Caderno e Notebook marcados	Resumo preserva os dois materiais.
Texto com sinais de tag no objetivo	Aparece como texto literal.
Tela estreita e teclado	Campos alcançáveis e foco visível.
Novo preenchimento	Regras e estado voltam à condição inicial.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual requisito você demonstrou com maior autonomia? Qual precisa corrigir?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

MDN: seu primeiro formulário

MDN: controles nativos de formulário (em inglês)

MDN: validação de formulários

MDN: atributo disabled (em inglês)

MDN: uso de FormData (em inglês)

W3C WAI: rótulos dos controles (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

AULA 09 · 18/12/2026

Corrigir, recuperar e concluir

Devolutiva, depuração e nova evidência de aprendizagem

Prof. Fabrício Rodrigues · Curso Técnico em Informática

O que vamos aprender

• Interpretar a devolutiva com base em critérios e testes.

• Localizar e corrigir defeitos de rótulo, coleta e estado.

• Demonstrar novamente os critérios pendentes.

Ponto de partida

Abra a base específica desta data em outra pasta. Retome o projeto e a explicação da aula anterior. Se não tiver os arquivos, baixe a base deste encontro: ela contém o ponto de partida necessário.

Devolutiva sem presumir os resultados

O professor devolve a avaliação real com os critérios e as evidências observadas. Não existe uma nota antecipada neste material. Escolha o que precisa retomar a partir dessa devolutiva.

O projeto de recuperação contém defeitos intencionais para investigar. Quem já demonstrou os critérios pode explicar os defeitos e ampliar os testes, conforme orientação do professor.

Investigação de rótulos e coleta

Clique no rótulo de contato e observe o foco. Compare for com o id. Depois confira se o campo possui name e se a coleta usa o formulário correto.

Faça uma mudança por vez e repita o mesmo teste. Um formulário vazio de dados no resumo pode indicar um problema de coleta, mesmo que os campos tenham sido preenchidos.

```
const dados = new FormData(form);
const contato = dados.get('contato');
const sintomas = dados.getAll('sintomas');
// getAll preserva várias opções com o mesmo name.
```

Estado e nova tentativa

Alterne a opção Outra oficina e confira visibilidade, obrigatoriedade e participação do tema sugerido. Compare as propriedades com a tabela de estados estudada.

Documente o problema, a hipótese, a alteração feita e o resultado depois da correção. Use uma tarefa equivalente para demonstrar os critérios ainda pendentes; siga as regras de recuperação da turma.

```
// hidden controla a apresentação do bloco.
// disabled retira o campo da validação e do FormData.
// required pede um valor quando o campo está ativo.
```

Construção passo a passo

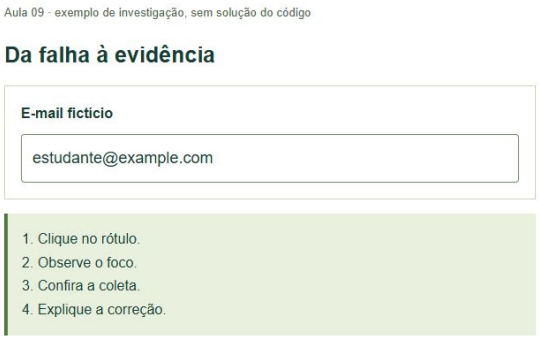
Salve com Ctrl+S e confira no navegador antes de avançar. Na aula 02, o CSS e o JavaScript são apoio pronto. Nas aulas 03 a 07, continue a mesma pasta do chamado de suporte. As bases por data permitem recuperar o ponto de partida.

1 · Investigar antes de alterar

Onde inserir: Abra a base de recuperação de 18/12 em uma pasta separada.

Entenda: Reproduza a falha; anote esperado e observado; localize a causa no código.

Resultado esperado: Cada correção tem um teste de regressão; aparência sozinha não comprova funcionamento.



Exemplo visual usado nesta etapa da aula.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

2 · Corrigir e demonstrar

Onde inserir: Ajuste os arquivos da recuperação e reteste os requisitos pendentes da avaliação.

Entenda: Mostre a relação entre label e id, a coleta por name e o estado do campo condicional.

Resultado esperado: Entregue ZIP e TXT com a correção explicada; combine a evidência com a devolutiva individual.

Salve e confira: Execute o caso correspondente na ficha de testes e registre o observado.

Prática individual

1. Ler a devolutiva e escolher os critérios a retomar com o professor.

2. Abrir a base de recuperação e investigar os defeitos de rótulo, name, coleta e campo condicional.

3. Corrigir uma hipótese por vez e repetir os testes.

4. Entregar a versão corrigida com o registro do progresso e explicar uma correção sem ler a solução.

Casos de teste

Caso	Resultado esperado após a correção
Clicar no rótulo de contato	O foco chega ao contato.
Preencher contato e gerar resumo	O valor informado aparece no resumo.
Escolher Outra oficina	Tema fica habilitado e obrigatório.
Sair de Outra oficina	Tema fica oculto e fora da coleta.
Corrigir um valor inválido	A próxima tentativa pode ocorrer.
Repetir o caso da avaliação	A evidência do critério pendente fica registrada.

Checklist

[ ] Implementei os requisitos da atividade desta aula.

[ ] Testei os casos da tabela e registrei o resultado observado.

[ ] Consigo explicar uma decisão e uma correção.

[ ] Guardei o ZIP e o TXT com nome, turma e aula.

Uma explicação para fechar

Qual era a causa do erro, como você a corrigiu e qual teste confirmou a mudança?

Entrega

Pasta com index.html, style.css e script.js em ZIP, acompanhada do registro de testes em TXT.

Abra o Registro de testes da data, informe nome e turma e baixe o TXT. Entregue ZIP com index.html, style.css e script.js, e o TXT, durante a aula. Não há envio automático. Não repita o diagnóstico de 16/10 nas aulas do módulo.

Referências para tirar dúvidas

W3C WAI: rótulos dos controles (em inglês)

W3C WAI: notificações e erros (em inglês)

MDN: uso de FormData (em inglês)

MDN: atributo disabled (em inglês)

MDN: setCustomValidity (em inglês)

VS Code: arquivos e pastas (em inglês)

Live Server de Ritwick Dey: abrir o projeto (em inglês)

Glossário

Elemento	Função
form	Agrupa os controles da tarefa.
label / for / id	Rótulo associado ao controle.
input / type	Campo e seu tipo de comportamento.
name / value	Nome do dado e valor do controle.
select / option	Lista e suas alternativas.
textarea	Texto de várias linhas.
fieldset / legend	Grupo relacionado e título do grupo.
required	Atributo que exige um valor.
FormData / getAll	Coleta por name e coleta de múltiplos valores.
hidden / disabled	Apresentação e participação do controle.

Fontes com links

Base anterior: portal do Prof. Ábner Lucas

MDN: seu primeiro formulário

MDN: tipos de input (em inglês)

W3C WAI: rótulos dos controles (em inglês)

W3C WAI: agrupamento de controles (em inglês)

MDN: validação de formulários

MDN: estilização de formulários (em inglês)

MDN: fundamentos de CSS Grid

MDN: evento submit (em inglês)

MDN: uso de FormData (em inglês)

MDN: setCustomValidity (em inglês)

W3C WAI: notificações e erros (em inglês)

MDN: envio de dados (em inglês)

MDN: elemento button (em inglês)

MDN: atributo disabled (em inglês)

GitHub: funcionamento do Pages

GitHub: configuração da publicação

MDN: textContent (em inglês)

VS Code: arquivos e pastas (em inglês)

- Live Server de Ritwick Dey: abrir o projeto (em inglês)
- MDN: controles nativos de formulário (em inglês)
- MDN: HTTPS (em inglês)